Notice explicative – La Dernière Lanterne

BUT MMI – R312 · Nom Prénom : .....

« La Dernière Lanterne » est un jeu de plate-forme en JavaScript avec Phaser 3. Il propose un **mode histoire** (quatre niveaux où il faut combattre rats et gobelins et rallumer trois lanternes pour sauver le village, plus une salle bonus cachée) et un **mode duo** en PvP (deux joueurs sur le même clavier, le premier à deux manches gagne).

Organisation du projet

- `index.html` : page d'accueil (menu, prologue, commandes, crédits) ; ses boutons lancent le jeu.
- `src/index.js` : configuration Phaser (`config`) et création du jeu avec `new Phaser.Game(config)`.
- `src/js/` : une classe de scène par fichier – `chargement`, `niveau`, `niveau1` à `niveau4`, `secret`, `pvp`, `hud`, `fin`.
- `src/assets/` : images, spritesheets, sons et cartes Tiled au format JSON.

Spécificités du code

- **Héritage entre niveaux.** `niveau.js` est une classe mère qui contient toute la logique commune (joueur, tir, ennemis, portes, items). Les quatre niveaux et la salle secrète en héritent et ne redéfinissent que leur carte, leur type d'item et `destinationPorte()`.
- **Chargement unique et données partagées.** La scène `chargement` charge tous les assets et crée les animations une seule fois. Les vies, le score et le temps sont stockés dans le `registry`, lu par toutes les scènes.
- **Affichage en scène parallèle.** `hud` est lancée par-dessus le niveau avec `scene.launch()` : sa caméra n'est pas zoomée, donc les textes restent fixes et nets alors que la caméra du niveau suit le joueur avec un zoom ×2.
- **Cartes Tiled.** Calques `background`, `plateform`, `ladder`, `decoration` et `death layer`. Le calque d'objets est lu par nom (`Spawn`, `Item`, `ennemi 1`, `ennemi 2`, `door`, `lanterne`) pour placer le joueur, les items, les ennemis, les portes et la lanterne.
- **Lanternes et progression.** Les niveaux 2, 3 et 4 contiennent chacun une lanterne éteinte et de plus en plus d'ennemis (11, 14 puis 17). La toucher lance son animation, rend ses vies au joueur et déverrouille la porte de sortie ; la troisième lanterne mène à la victoire.
- **Tir et cibles.** Le joueur porte un attribut `direction` ; `tirer()` crée une balle dans un groupe avec un `coefDir`. Une superposition balle/ennemi appelle `hit()`, qui retire un point de vie et détruit l'ennemi à 0.
- **Ennemis.** Le rat patrouille, fait demi-tour au bord des plates-formes et charge le joueur proche. Le gobelin lance un caillou à intervalle régulier grâce à un timer récurrent.
- **Timers.** `delayedCall()` pour le délai entre deux tirs, l'invincibilité après un coup, la réapparition et l'apparition des items ; `addEvent()` en boucle pour le chronomètre, l'attaque du gobelin et le cristal de soin ; à répétitions fixes pour le compte à rebours du duo.
- **Bords du monde et acide.** L'événement `worldbounds` détruit les balles perdues et détecte une chute : le joueur perd une vie et est replacé au départ. Les tuiles d'acide sont testées avec `getTileAtWorldXY()`.
- **Portes.** Sprite statique avec animation d'ouverture, déclenchée par `overlap` et la touche E, puis changement de scène. Un cristal protecteur révèle un chemin caché jusqu'à une porte secrète, qui mène à la salle bonus.
- **Second joueur.** En duo, la même fonction `deplacerJoueur()` sert aux deux joueurs : seul le jeu de touches change. Chaque joueur a sa barre de vie, sa couleur de balle et son sprite.
- **Son.** Musique de fond en boucle et bruitages (tir, saut, coup, porte, item, victoire).

Commandes

	Se déplacer	Sauter	Tirer	Échelle	Porte
Joueur 1	Q / D	Espace	H	Z / S	E
Joueur 2 (duo)	← / →	2	1	–	–

Lancement

Le code est réparti en modules : le jeu doit être servi par un serveur local, par exemple l'extension **Live Server** de VS Code (clic droit sur `index.html`, « Open with Live Server »). Phaser est chargé depuis un CDN, une connexion internet est donc nécessaire. Échap ramène au menu.